

Sabre & Neural Planning

Lara J. Martin (she/they)

<https://laramartin.net/neurosymbolic-text-gen/>

Sabre slides from Stephen Ware

Learning Objectives

Explore how Theory of Mind can expand narrative planning in the Sabre planner

Distinguish between types of neural planning

Review: Planning

Planning: The process of **searching** for a plan

- This is why we can use algorithms like BFS to find plans
- “Plain” state-based search is useful when we just want to get to the goal (efficiently); planning is useful when we care about the *path*
- What we think of as “planning” is a combination of search and logic

Plan: The result of planning; a sequence of steps from the initial state to a goal state

Policy: A collection of transition functions ($\text{Result}(s, a) \rightarrow s_r$) that tell the agent what action it should take for a given state

- This will become more relevant when talking about reinforcement learning

Review: What are some of the components of PDDL?

Review: Heuristics for Planning

Neither forward nor backward search is efficient without good **heuristics**.

In search, a heuristic function $h(s)$ estimates the distance from a state to the goal.

Admissible heuristics never over-estimate the true distance and can be used with **A* search** to find optimal solutions.

Admissible heuristics can be derived from a **relaxed problem** (approximation) that is easier to solve.

The “**ignore preconditions**” heuristic relaxes the problem.

Sabre

A Narrative Planner Supporting Intention and Deep Theory of Mind

Stephen G. Ware

Cory Siler



Narrative
Intelligence
LAB



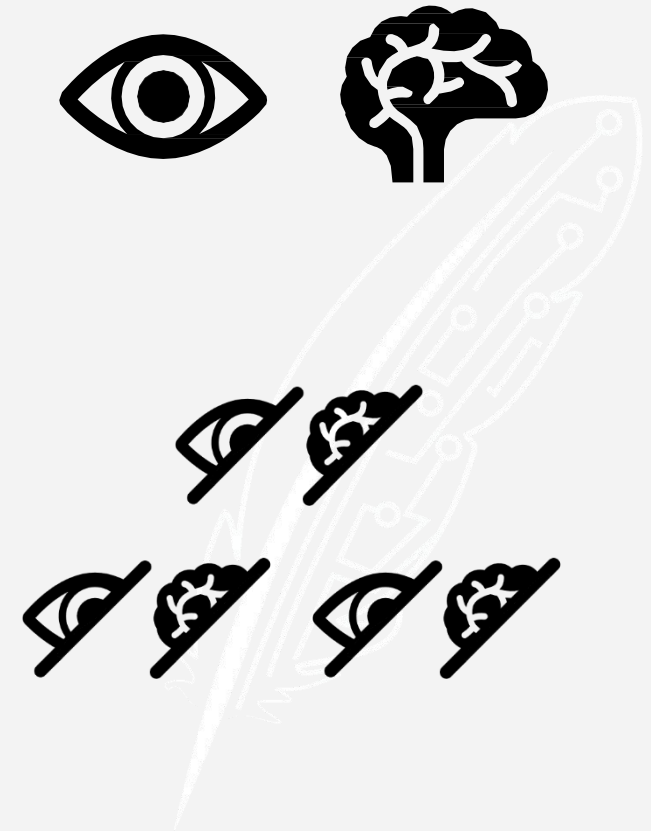
University of
Kentucky

Sabre + Neural Planning

Narrative Planning

A single decision maker

creates the appearance
of a multi-agent system.

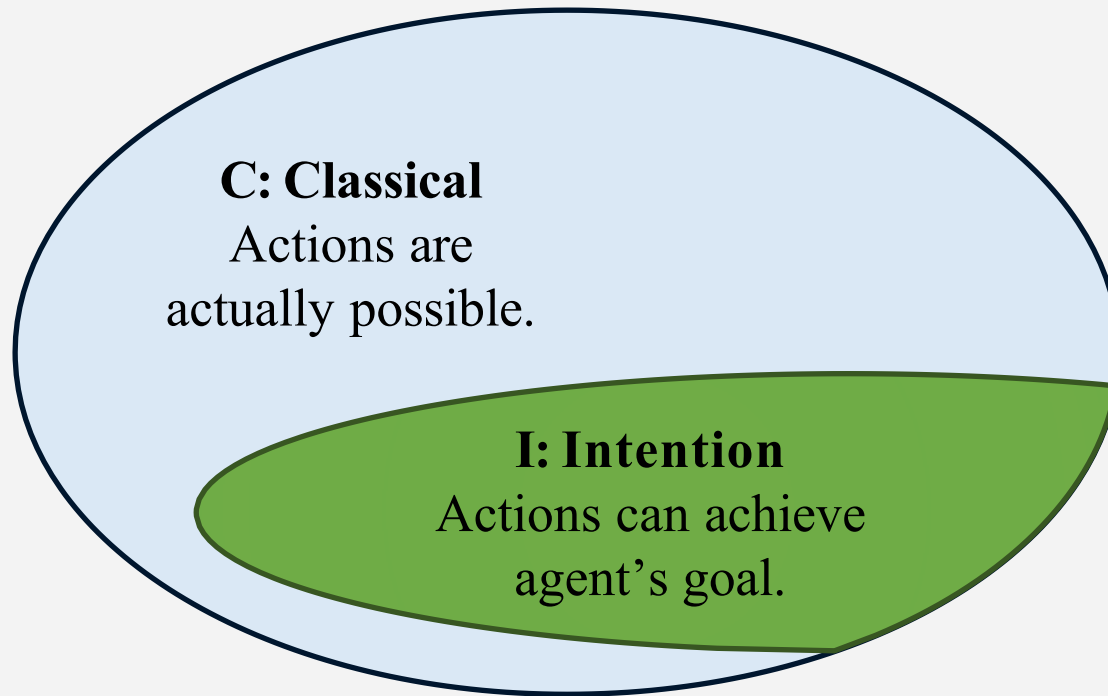


Intentions and Beliefs

C: Classical
Actions are
actually possible.

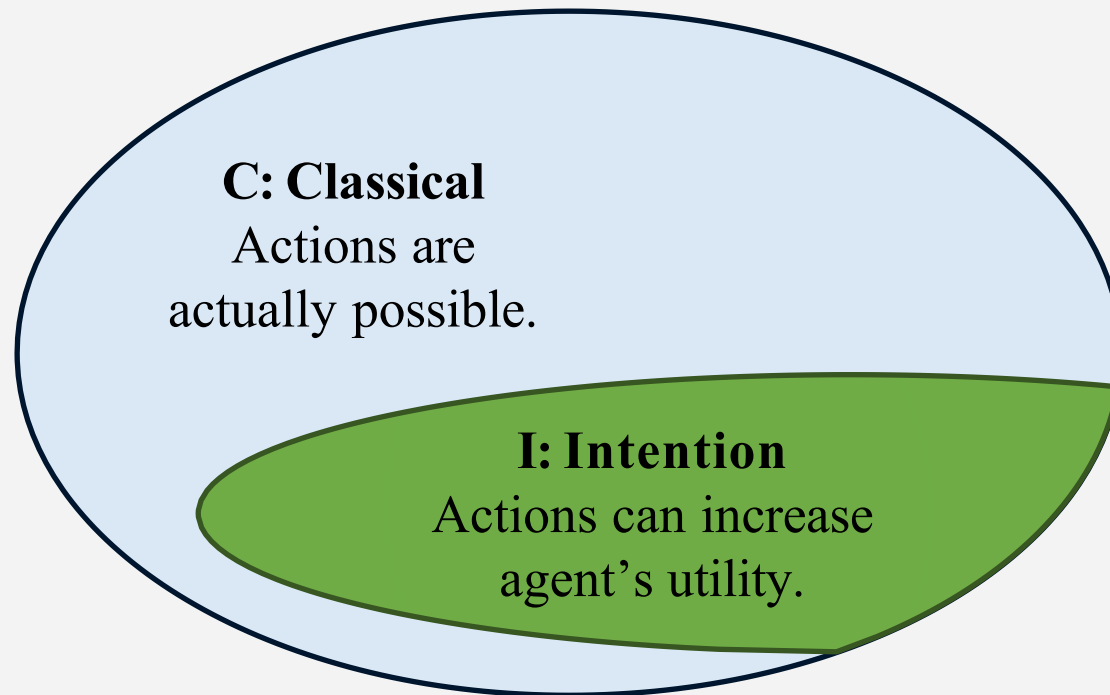


Intentions and Beliefs

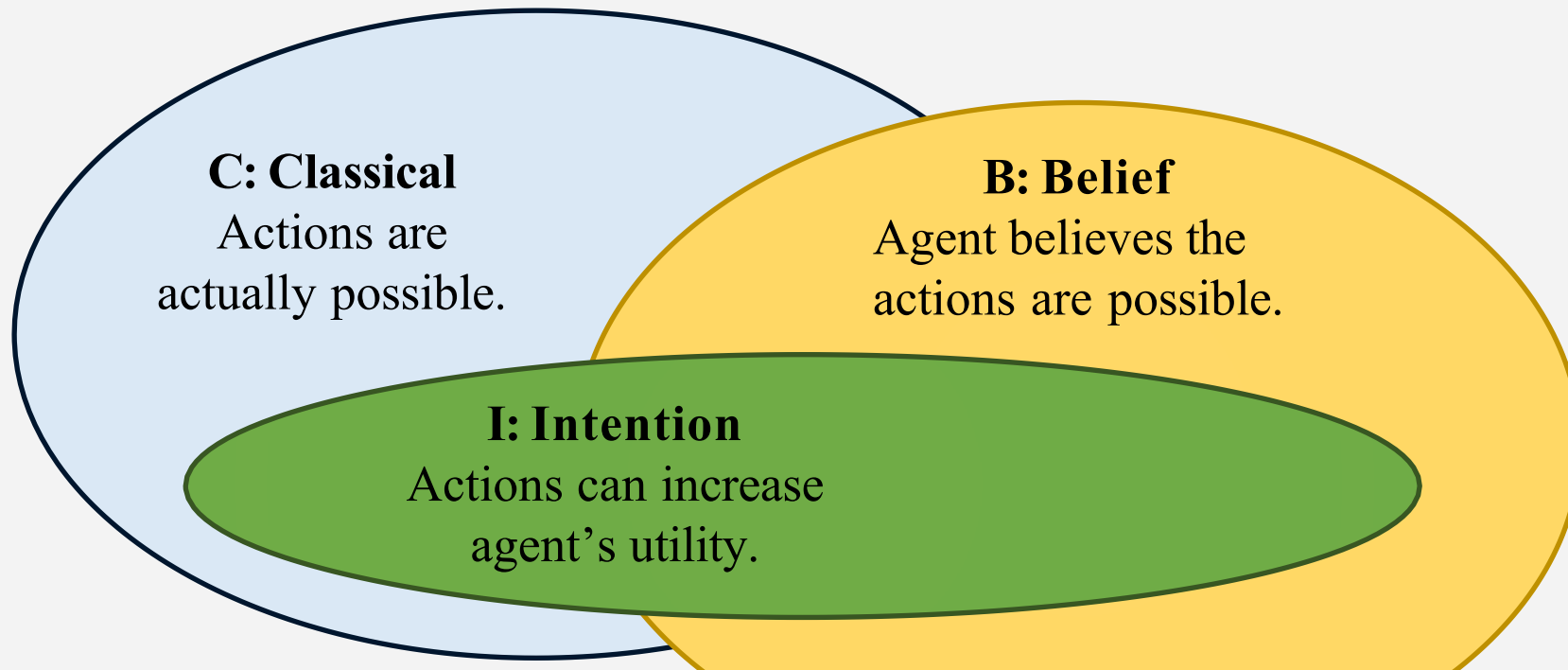


- Riedl and Young, “Narrative planning: balancing plot and character,” in JAIR 2010
- Teutenberg and Porteous, “Efficient intent-based narrative generation...,” in AAMAS 2013
- Ware and Young, “Glaive: a state-space narrative planner...,” in AIIDE 2014

Intentions and Beliefs

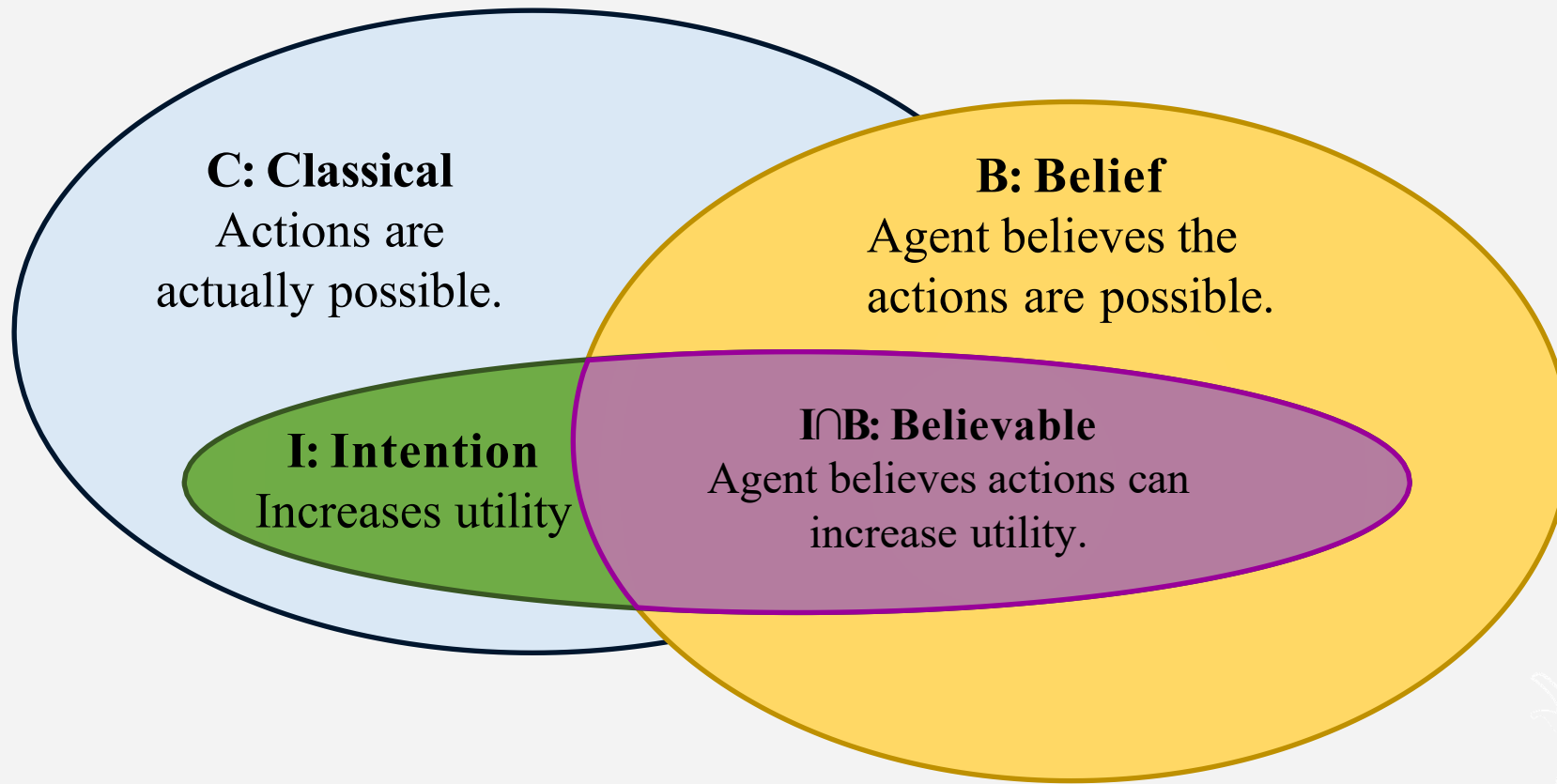


Intentions and Beliefs



- Eger and Martens, “Character beliefs in story generation,” INT 2017
- Thorne and Young, “Generating stories ...by modeling false character beliefs,” in INT 2017
- Shirvani, Ware, and Farrell, “A possible worlds model of belief...,” in AIIDE 2017

Intentions and Beliefs



- Shirvani, Farrell, and Ware, “Combining intentionality and belief...,” in AIIDE 2018

Syntax and Features

Fluents

$at(Tom) =$



Helmert, “The Fast Downward planning system,” in JAIR 2006

Fluents

$at(Tom) = \textit{Cottage}$



Helmert, “The Fast Downward planning system,” in JAIR 2006

Fluents

$at(Tom) = Cottage$

$path(Cottage, Market) = T$



Fluents

$at(Tom) = Cottage$

$path(Cottage, Market) = T$

$wealth(Merchant) = 3$



Fluents

$at(Tom) = Cottage$

$path(Cottage, Market) = T$

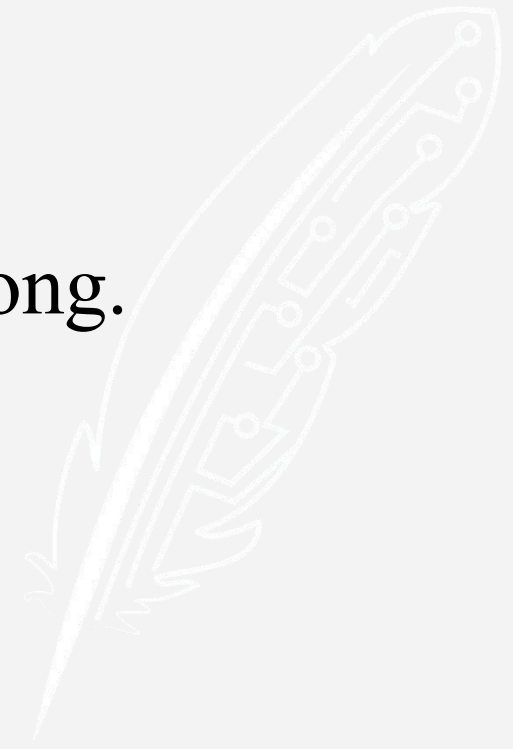
$wealth(Merchant) = 3$

$believes(Tom, wealth(Merchant)) = 2$



Theory of Mind

- Arbitrarily deep
what x believes y believes z believes...
- No uncertainty
Everyone commits to beliefs, which can be wrong.



Other Syntactical Features

- Negation
- Disjunction
- Conditional Effects
- First Order Quantifiers



Actions

buy(Tom, Potion, Merchant, Market)



Actions

a: buy(Tom, Potion, Merchant, Market)



Actions

a: buy(Tom, Potion, Merchant, Market)

PRE(*a*):



Actions

a: buy(Tom, Potion, Merchant, Market)

PRE(*a*): *at(Tom) = Market*



Actions

a: buy(Tom, Potion, Merchant, Market)

PRE(*a*): *at(Tom) = Market* $\wedge$ *at(Merchant) = Market*



Actions

a: buy(Tom, Potion, Merchant, Market)

PRE(*a*): *at(Tom) = Market* $\wedge$ *at(Merchant) = Market* $\wedge$
at(Potion) = Merchant



Actions

a: buy(Tom, Potion, Merchant, Market)

$\text{PRE}(a): \text{at}(\text{Tom}) = \text{Market} \wedge \text{at}(\text{Merchant}) = \text{Market} \wedge$
 $\text{at}(\text{Potion}) = \text{Merchant} \wedge \text{wealth}(\text{Tom}) \geq 1$

Actions

a: buy(Tom, Potion, Merchant, Market)

PRE(*a*): *at(Tom) = Market $\wedge$ at(Merchant) = Market $\wedge$
at(Potion) = Merchant $\wedge$ wealth(Tom) ≥ 1*

EFF(*a*):

Actions

a: buy(Tom, Potion, Merchant, Market)

PRE(*a*): *at(Tom) = Market $\wedge$ at(Merchant) = Market $\wedge$
at(Potion) = Merchant $\wedge$ wealth(Tom) $\geq$ 1*

EFF(*a*): *at(Potion) = Tom*

Actions

a: buy(Tom, Potion, Merchant, Market)

$\text{PRE}(a): at(Tom) = Market \wedge at(Merchant) = Market \wedge$
 $at(Potion) = Merchant \wedge wealth(Tom) \geq 1$

$\text{EFF}(a): at(Potion) = Tom \wedge \text{wealth}(Merchant) += 1$

Actions

a: buy(Tom, Potion, Merchant, Market)

$\text{PRE}(a): at(Tom) = Market \wedge at(Merchant) = Market \wedge$
 $at(Potion) = Merchant \wedge wealth(Tom) \geq 1$

$\text{EFF}(a): at(Potion) = Tom \wedge wealth(Merchant) += 1 \wedge$
 $wealth(Tom) -= 1$

Actions

a: buy(Tom, Potion, Merchant, Market)

$\text{PRE}(a): at(Tom) = Market \wedge at(Merchant) = Market \wedge$
 $at(Potion) = Merchant \wedge wealth(Tom) \geq 1$

$\text{EFF}(a): at(Potion) = Tom \wedge wealth(Merchant) += 1 \wedge$
 $wealth(Tom) -= 1$

$\text{CON}(a):$

Actions

a: buy(Tom, Potion, Merchant, Market)

$\text{PRE}(a): at(Tom) = Market \wedge at(Merchant) = Market \wedge$
 $at(Potion) = Merchant \wedge wealth(Tom) \geq 1$

$\text{EFF}(a): at(Potion) = Tom \wedge wealth(Merchant) += 1 \wedge$
 $wealth(Tom) -= 1$

$\text{CON}(a): \{Tom, Merchant\}$

Actions

a: buy(Tom, Potion, Merchant, Market)

$\text{PRE}(a): at(Tom) = Market \wedge at(Merchant) = Market \wedge$
 $at(Potion) = Merchant \wedge wealth(Tom) \geq 1$

$\text{EFF}(a): at(Potion) = Tom \wedge wealth(Merchant) += 1 \wedge$
 $wealth(Tom) -= 1$

$\text{CON}(a): \{Tom, Merchant\}$

$\text{OBS}(a, c):$

Actions

a: buy(Tom, Potion, Merchant, Market)

$\text{PRE}(a): at(Tom) = Market \wedge at(Merchant) = Market \wedge$
 $at(Potion) = Merchant \wedge wealth(Tom) \geq 1$

$\text{EFF}(a): at(Potion) = Tom \wedge wealth(Merchant) += 1 \wedge$
 $wealth(Tom) -= 1$

$\text{CON}(a): \{Tom, Merchant\}$

$\text{OBS}(a, c): at(c) = Market$

Triggers

t: see(Tom, Merchant, Market)

PRE(*t*):

EFF(*t*):



Triggers

t: see(Tom, Merchant, Market)

PRE(*t*): *at(Tom) = Market*

EFF(*t*):



Triggers

t: see(Tom, Merchant, Market)

PRE(*t*): *at(Tom) = Market* $\wedge$ *at(Merchant) = Market*

EFF(*t*):



Triggers

t: see(Tom, Merchant, Market)

PRE(*t*): *at(Tom) = Market* $\wedge$ *at(Merchant) = Market* $\wedge$
believes(Tom, at(Merchant)) $\neq$ Market

EFF(*t*):



Triggers

t: see(Tom, Merchant, Market)

PRE(*t*): *at(Tom) = Market* $\wedge$ *at(Merchant) = Market* $\wedge$
believes(Tom, at(Merchant)) $\neq$ *Market*

EFF(*t*): *believes(Tom, at(Merchant)) = Market*

Pre-Processing

- Make action and trigger results explicit
- Detect and remove immutable fluents
- Detect and remove impossible actions and triggers



Results of an Event

After Tom buys the potion from the merchant...

- Tom has the potion.
- Tom knows he has the potion.
- The merchant knows Tom has the potion.
- Tom know that the merchant knows that he has the potion.
- ...and so on.



Example Trigger: Two-Way Paths

t: add_path(y, x)

$\text{PRE}(t) \text{ path}(x, y) = \top \wedge \text{path}(y, x) = \perp$

$\text{EFF}(t): \text{path}(y, x) = \top$



Example Trigger: Two-Way Paths

t: add_path(Market, Cottage)

PRE(t): path(Cottage, Market) = $\top$ $\wedge$

path(Market, Cottage) = $\perp$

EFF(t): path(Market, Cottage) = $\top$



Example Action: Walk

a: walk(Tom, Market, Cottage)

$\text{PRE}(a): at(Tom) = Market \wedge$

$path(Market, Cottage) = \top$

$\text{EFF}(a): at(Tom) = Cottage$

$\text{CON}(a): \{Tom\}$

$\text{OBS}(a, c): at(c) = Market \vee at(c) = Cottage$



Example Action: Walk

a: walk(Tom, Market, Cottage)

PRE(*a*): *at(Tom) = Market* $\wedge$

path(Market, Cottage) = T

EFF(*a*): *at(Tom) = Cottage*

CON(*a*): {*Tom*}

OBS(*a, c*): *at(c) = Market* $\vee$ *at(c) = Cottage*



Example Action: Walk

a: walk(Tom, Market, Cottage)

PRE(*a*): *at(Tom) = Market*

EFF(*a*): *at(Tom) = Cottage*

CON(*a*): {*Tom*}

OBS(*a, c*): *at(c) = Market* $\vee$ *at(c) = Cottage*



Search

Algorithm 1 The Sabre algorithm

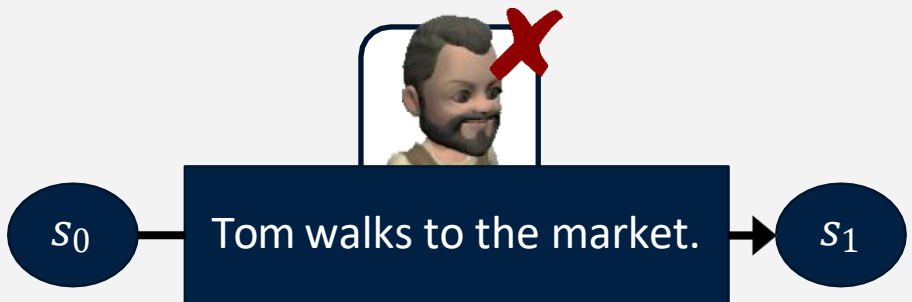
```
1: Let  $\mathcal{A}$  be the set of all actions defined in the domain.
2: SABRE( $c_{author}, s_0, \emptyset, s_0$ )
3: function SABRE( $c, r, \pi, s$ )
4:   Input: character  $c$ , start state  $r$ , plan  $\pi$ , current state  $s$ 
5:   if  $u(c, s) > u(c, r)$  and  $\pi$  is non-redundant then
6:     return  $\pi$ 
7:   Choose an action  $a \in \mathcal{A}$  such that  $s \models \text{PRE}(a)$ .
8:   for all  $c' \in \text{CON}(a)$  such that  $c' \neq c$  do
9:     Let state  $b = \alpha(a, \beta(c', s))$ .
10:    if  $b$  is undefined then return failure.
11:    else if SABRE( $c', b, \emptyset, b$ ) fails then return failure.
12:  return SABRE( $c, r, \pi \cup a, \alpha(a, s)$ )
```

S0

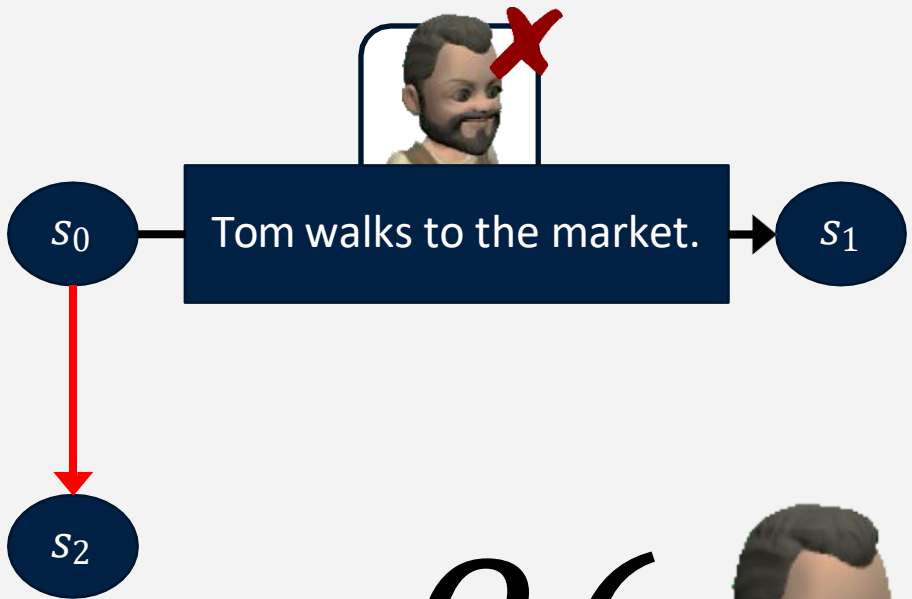


S_0

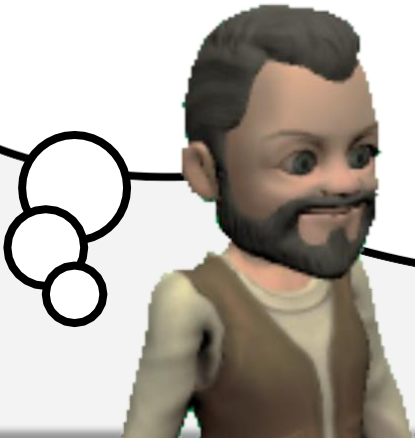


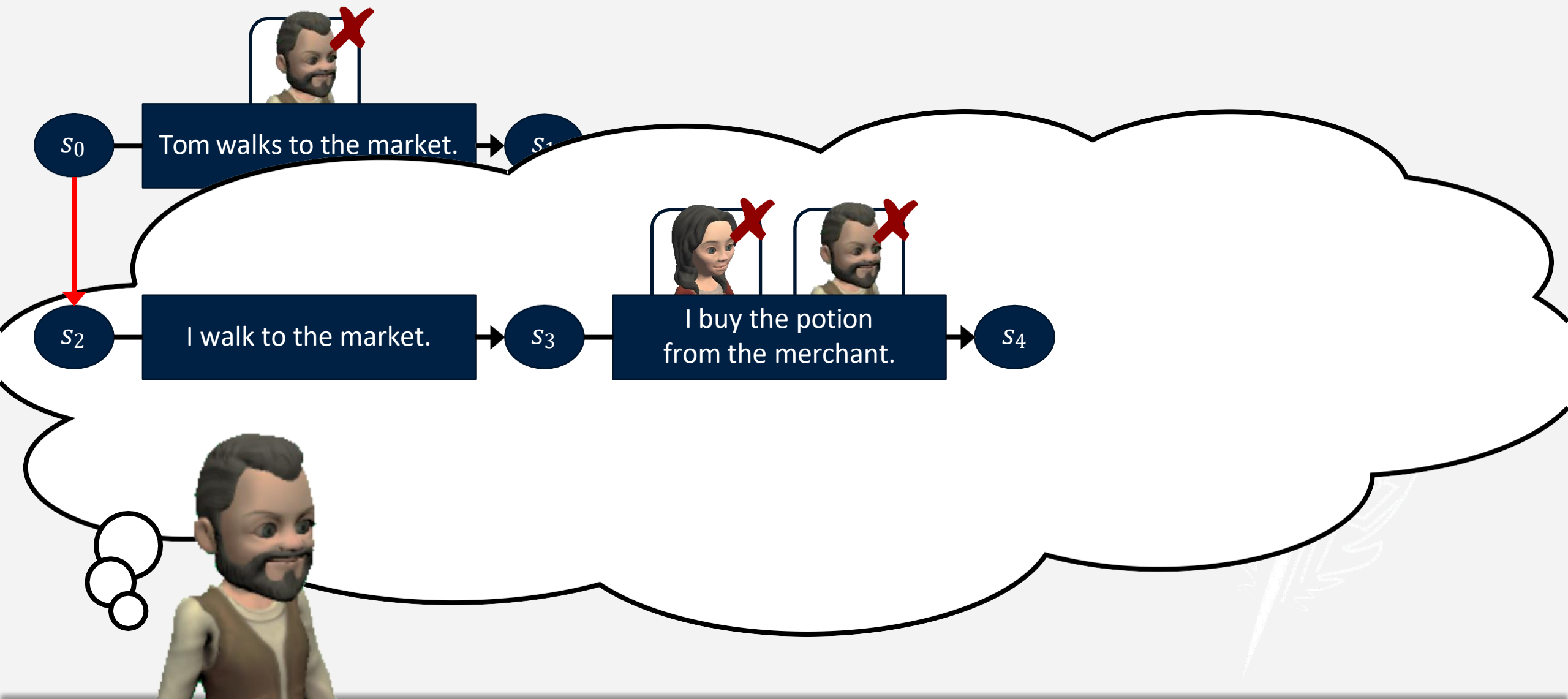


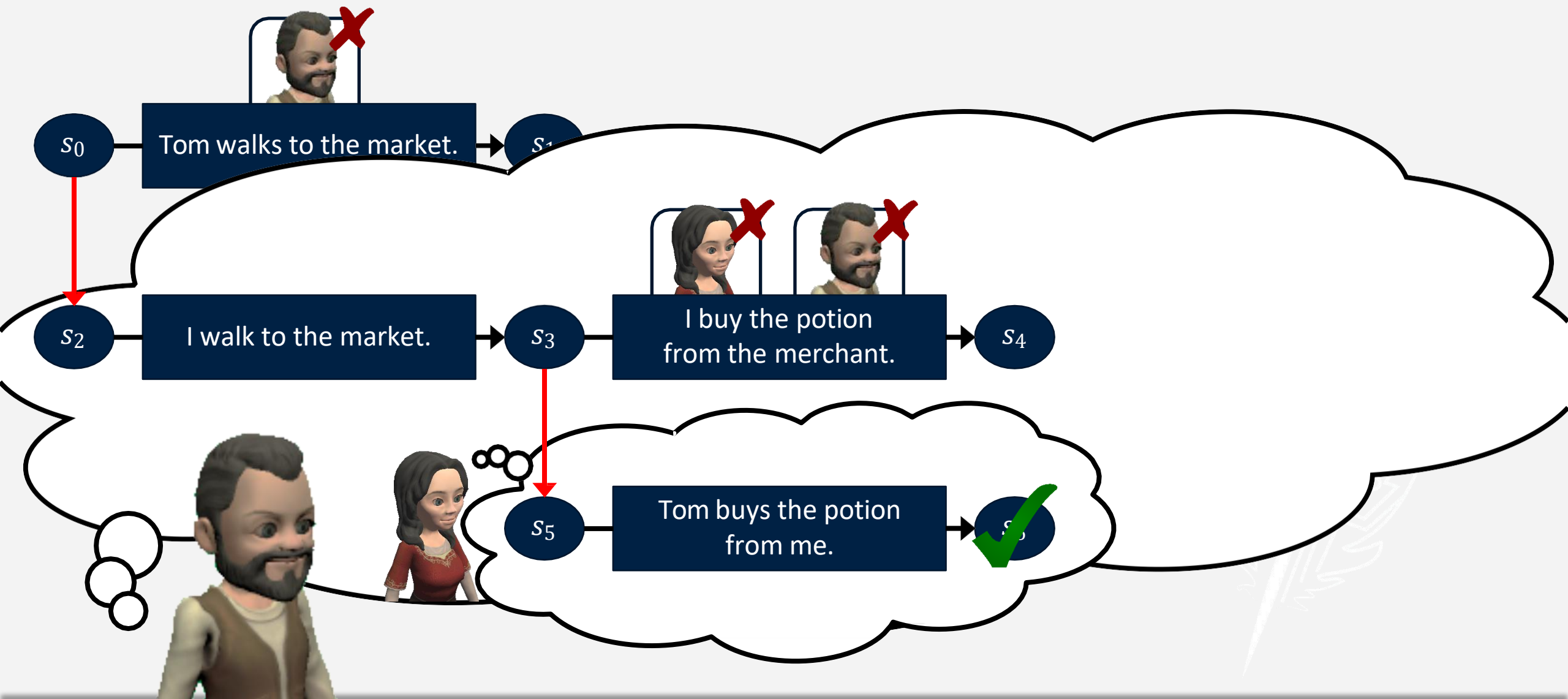
$$\alpha(\text{Tom walks to the market.}, s_0) = s_1$$

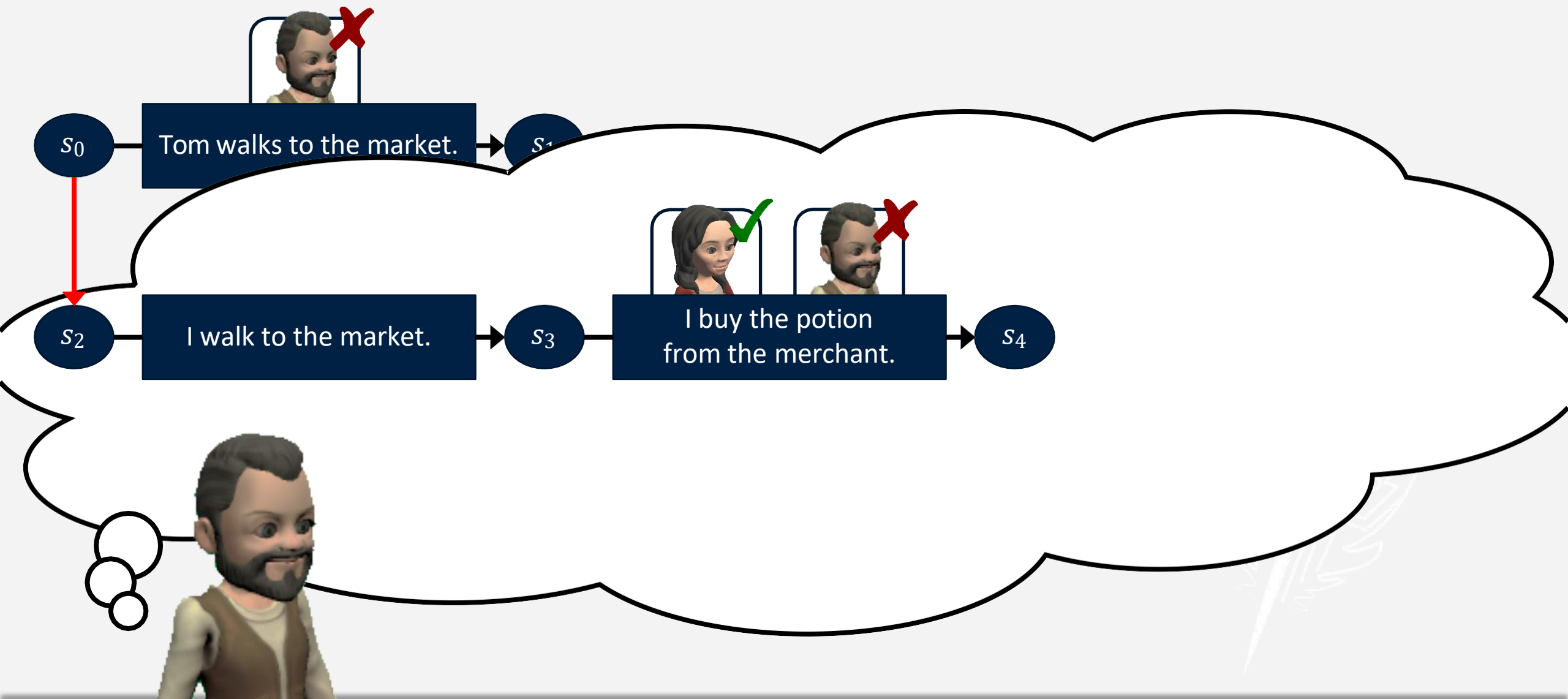


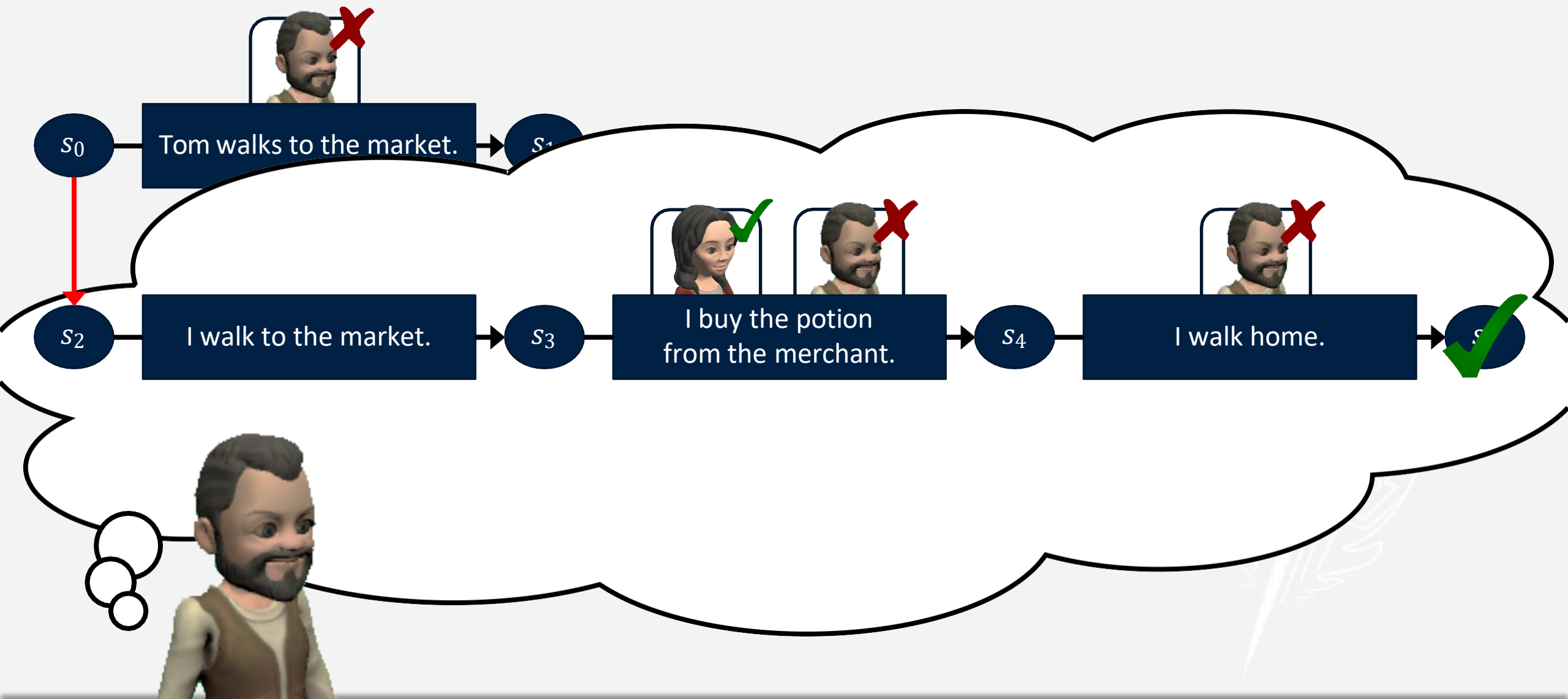
$$\beta(\text{Tom}, s_0) = s_2$$

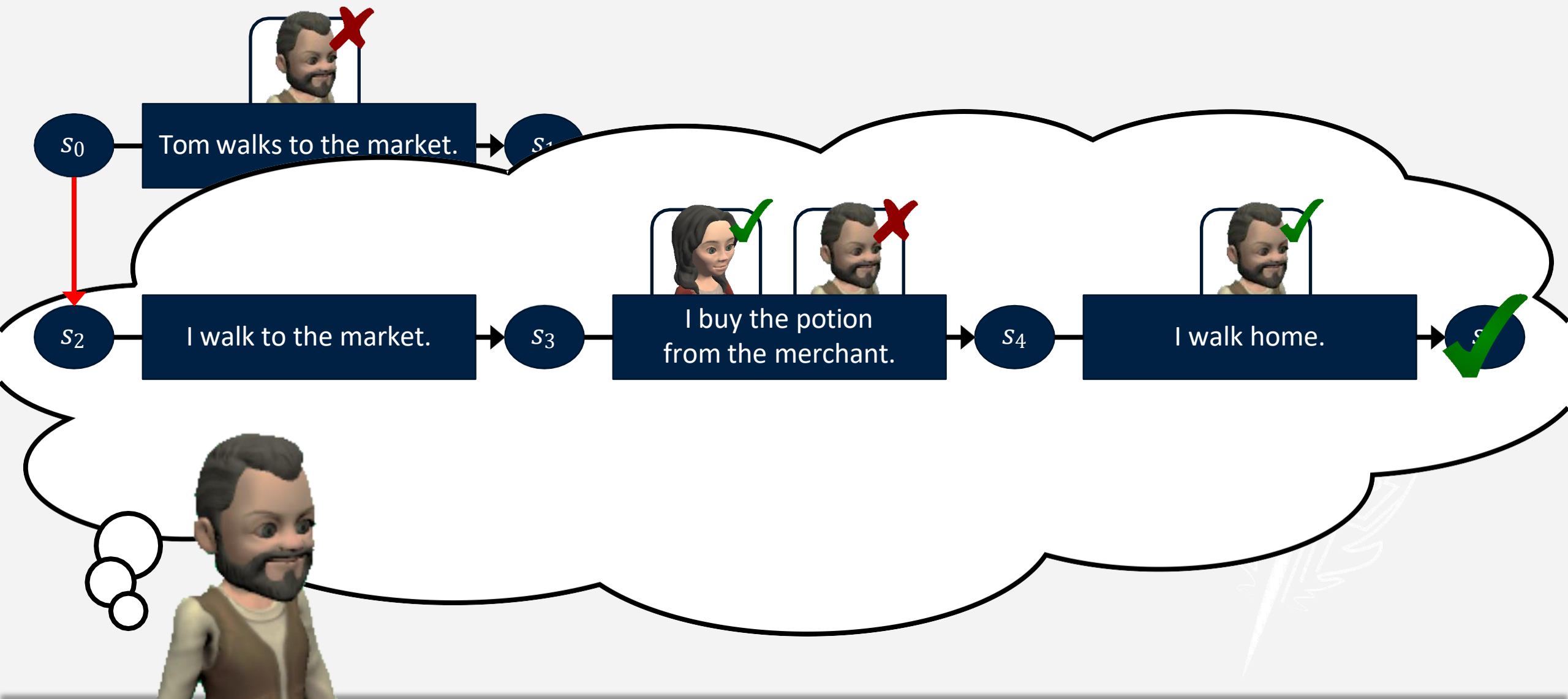


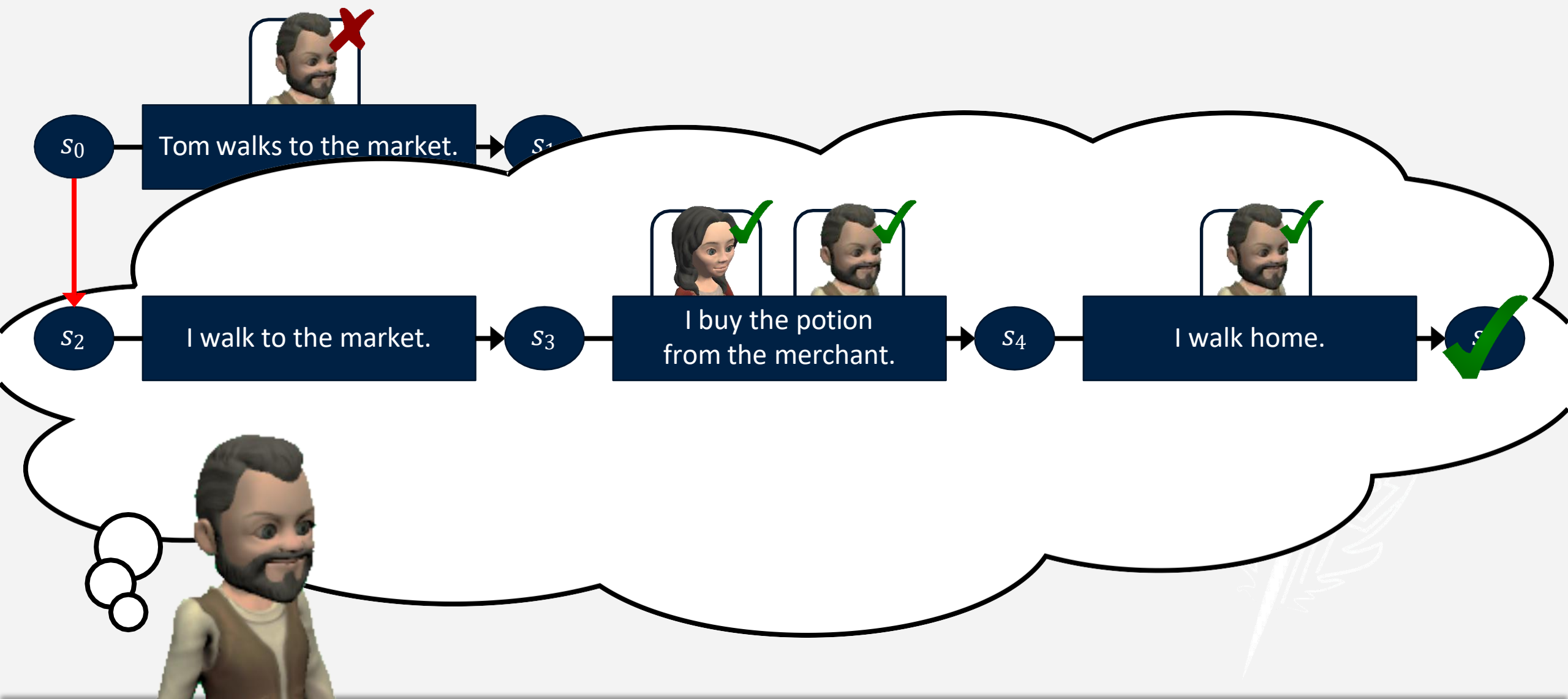


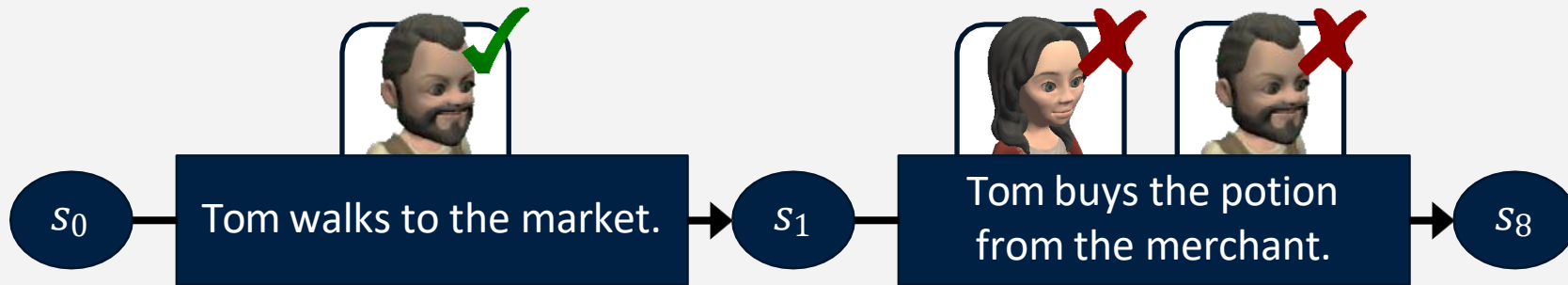


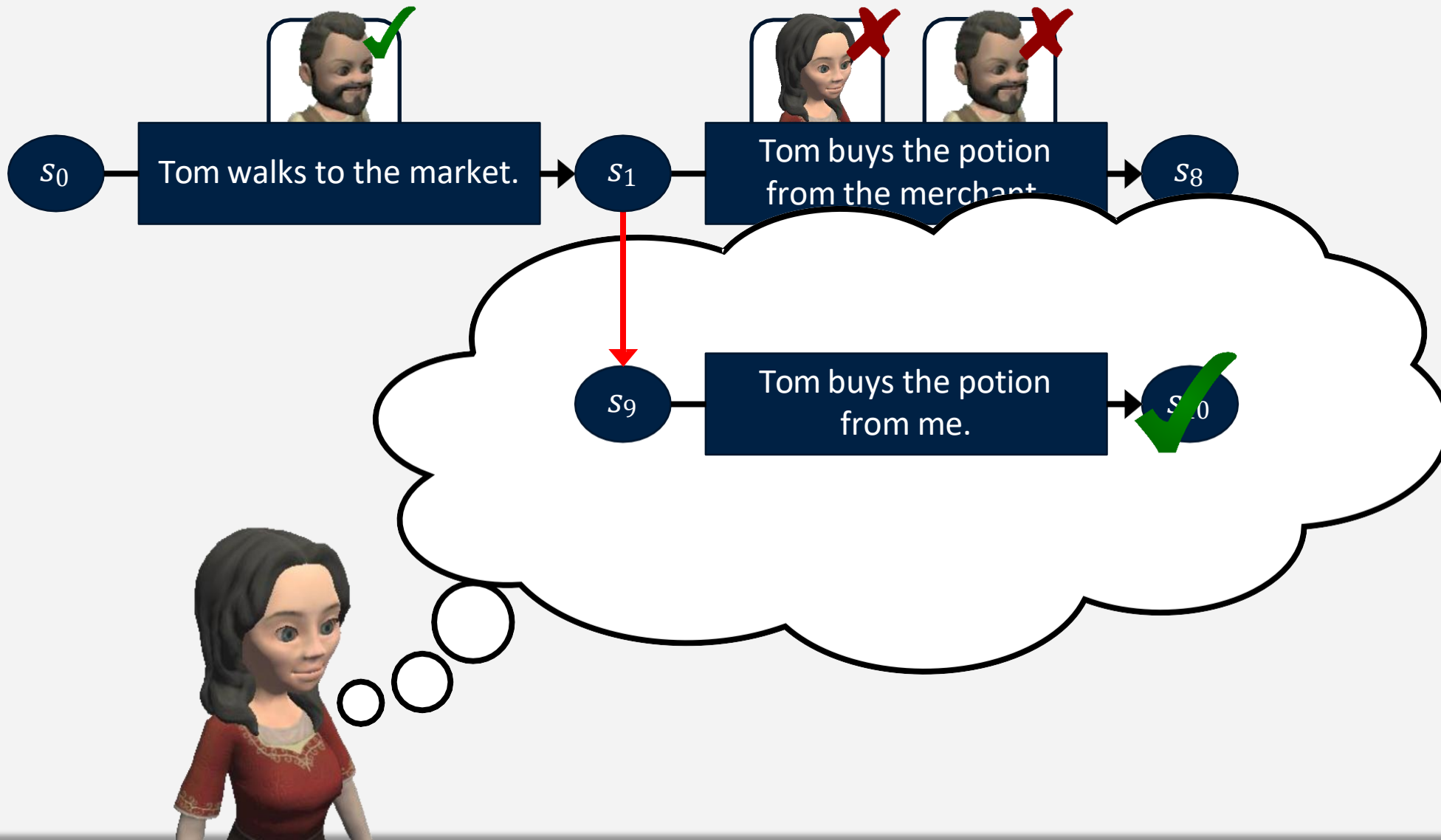


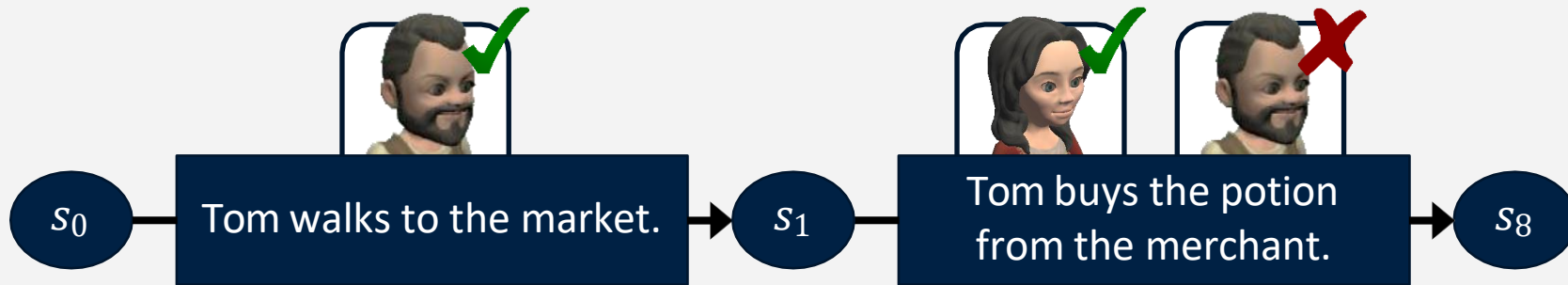


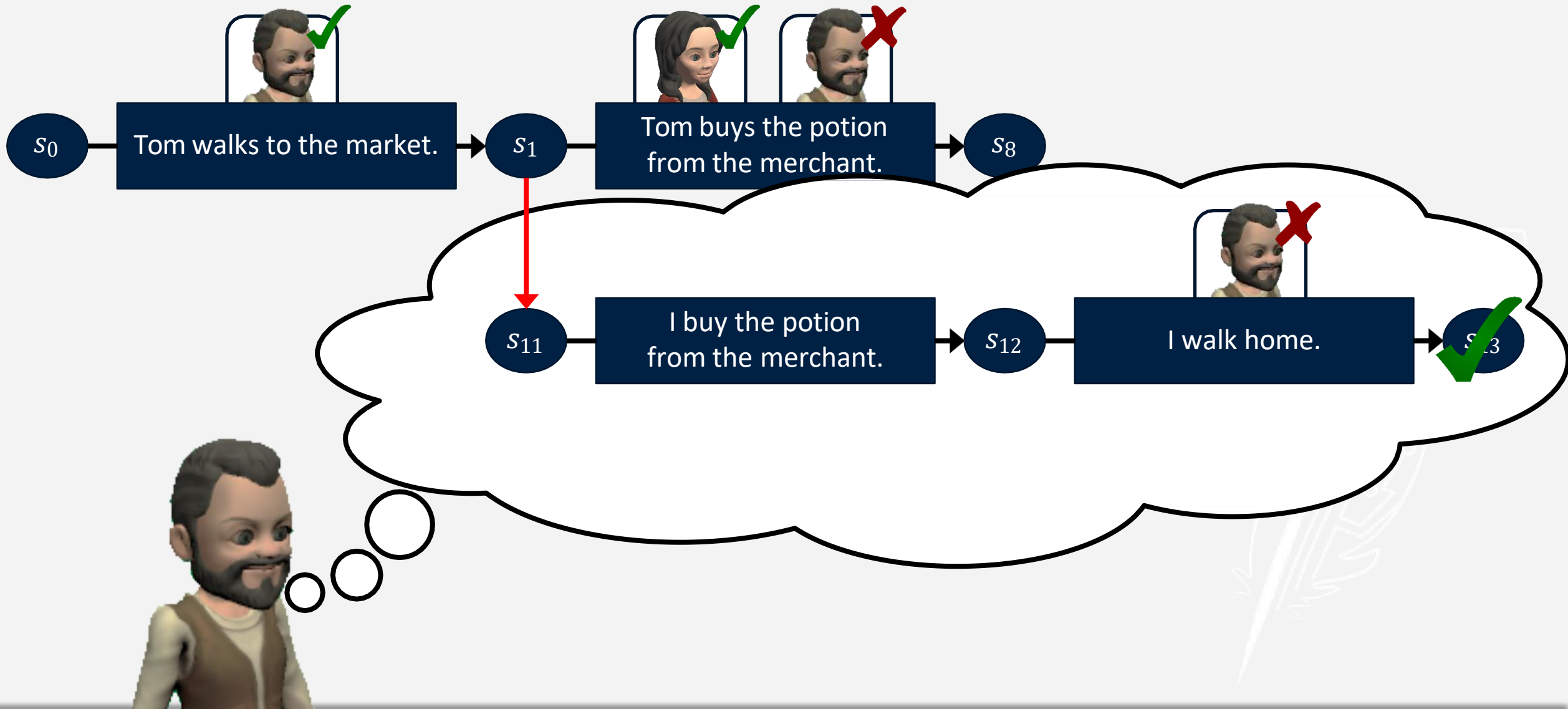


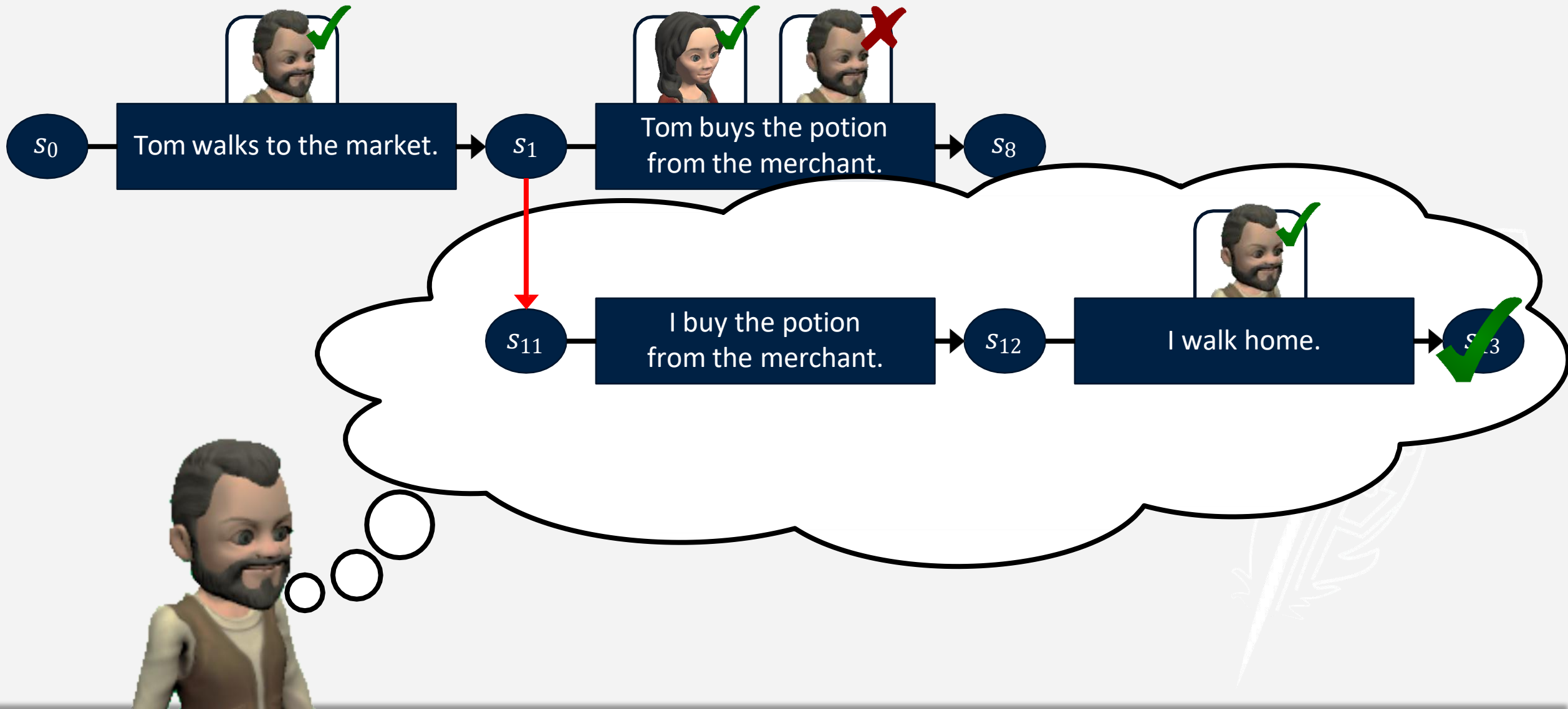


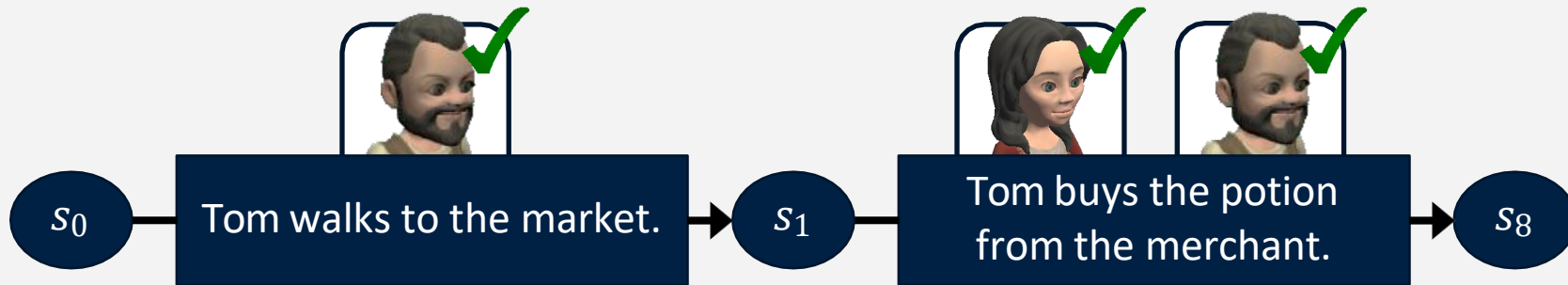




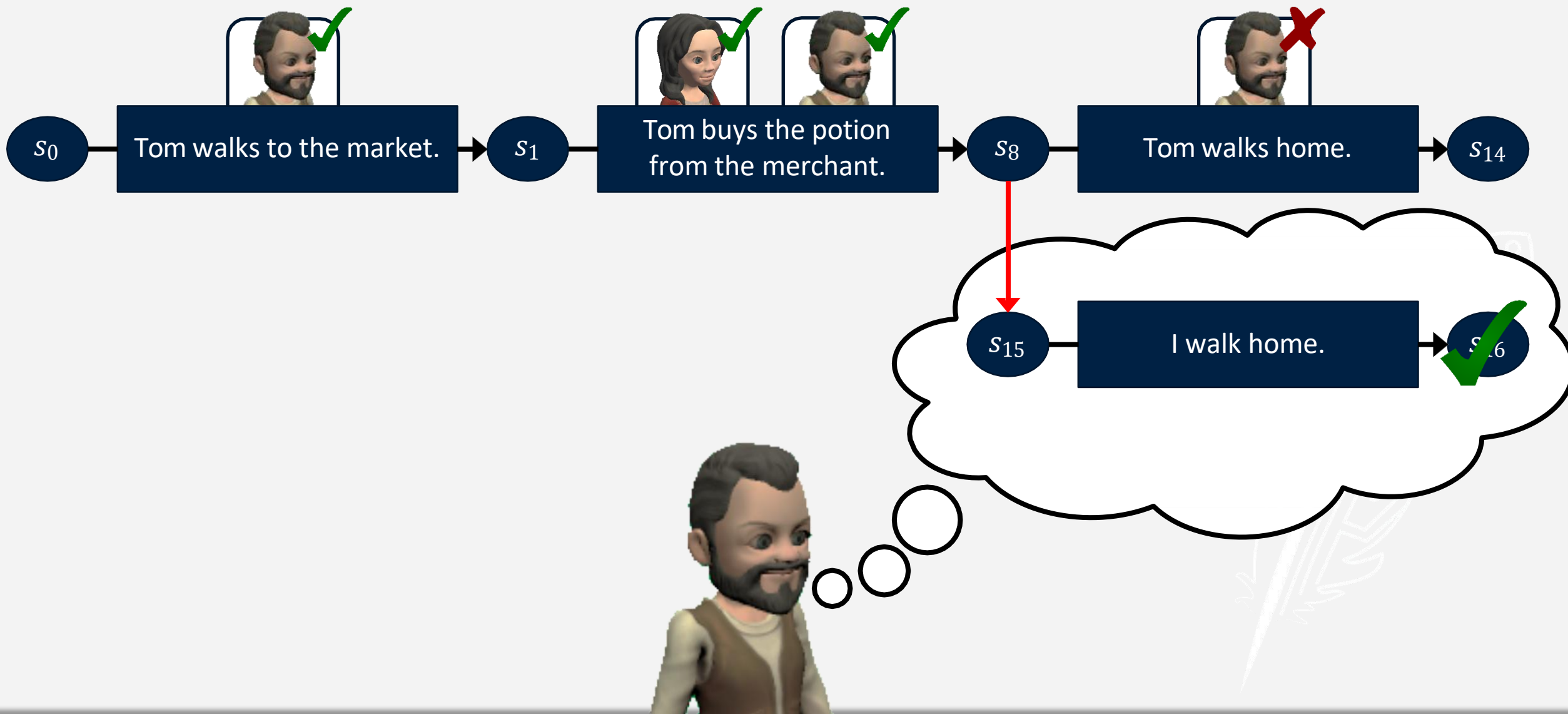














Evaluation

Comparing Sabre to Other Planners

	Centralized	Intentions	Beliefs	Uncertainty
Sabre	✓	✓	✓	✗

Comparing Sabre to Other Planners

	Centralized	Intentions	Beliefs	Uncertainty
Sabre	✓	✓	✓	✗
Glaive	✓	✓	✗	✗

- Riedl and Young, “Narrative planning: balancing plot and character,” in JAIR 2010
- Ware and Young, “CPOCL: a narrative planner supporting conflict,” in AIIDE 2011
- Teutenberg and Porteous, “Efficient intent-based narrative generation...,” in AAMAS 2013
- Ware and Young, “Glaive: a state-space narrative planner...,” in AIIDE 2014

Comparing Sabre to Other Planners

	Centralized	Intentions	Beliefs	Uncertainty
Sabre	✓	✓	✓	✗
Glaive	✓	✓	✗	✗
HeadSpace	✓	✗	~✓	✗

- Thorne and Young, “Generating stories ...by modeling false character beliefs,” in INT 2017

Comparing Sabre to Other Planners

	Centralized	Intentions	Beliefs	Uncertainty
Sabre	✓	✓	✓	✗
Glaive	✓	✓	✗	✗
HeadSpace	✓	✗	~✓	✗
IMPRACTical	✓	✓	~✓	✗

- Teutenberg and Porteous, “Incorporating global and local knowledge...,” in AAMAS 2015

Comparing Sabre to Other Planners

	Centralized	Intentions	Beliefs	Uncertainty
Sabre	✓	✓	✓	✗
Glaive	✓	✓	✗	✗
HeadSpace	✓	✗	~✓	✗
IMPRACTical	✓	✓	~✓	✗
Thespian	✗	✓	✓	✓

- Ryan, Summerville, Mateas, and Wardrip-Fruin, “Toward characters who observe...,” in EXAG 2015
- Si and Marsella, “Encoding Theory of Mind in character design...,” in AHCI 2014

Comparing Sabre to Other Planners

	Centralized	Intentions	Beliefs	Uncertainty
Sabre	✓	✓	✓	✗
Glaive	✓	✓	✗	✗
HeadSpace	✓	✗	~✓	✗
IMPRACTical	✓	✓	~✓	✗
Thespian	✗	✓	✓	✓
Ostari	✓	✓	✓	✓

- Eger and Martens, “Practical specification of belief manipulation in games,” in AIIDE 2017

Test Problems

- *Raiders*
- *Space*

- Ware and Young, “Glaive: a state-space narrative planner...,” in AIIDE 2014



Test Problems

- *Raiders*
- *Space*
- *Treasure*
- *Lovers*
- *Hubris*

- Farrell and Ware, “Narrative planning for belief and intention recognition,” in AIIDE 2020
- Shirvani, Farrell, and Ware, “Combining intentionality and belief ...,” in AIIDE 2018
- Christensen, Nelson, and Cardona-Rivera, “Using domain compilation to add belief ...,” in AIIDE 2020

Test Problems

- *Raiders*
- *Space*
- *Treasure*
- *Lovers*
- *Hubris*
- *BearBirdJr*

- Sack, “Micro-TaleSpin, a story generator,” 1992
- Meehan, “TALE-SPIN, an interactive program that writes stories,” in AAAI 1977



Test Problems

- *Raiders*
- *Space*
- *Treasure*
- *Lovers*
- *Hubris*
- *BearBirdJr*
- *Grandma*



- Ware, Garcia, Shirvani, and Farrell, “Multi-agent experience management ...,” in AIIDE 2019

Results

Domain	Nodes Generated	Time
<i>Raiders</i>	17,815	1.4 s
<i>Space</i>	192	6 ms
<i>Treasure</i>	288	1 ms
<i>Lovers</i>	5,198,414	40.3 m
<i>Hubris</i>	831	47 ms
<i>BearBirdJr</i>	34,084,068	14.0 m
<i>Grandma</i>	105,178,466	6.2 h

Conclusion

Limitations

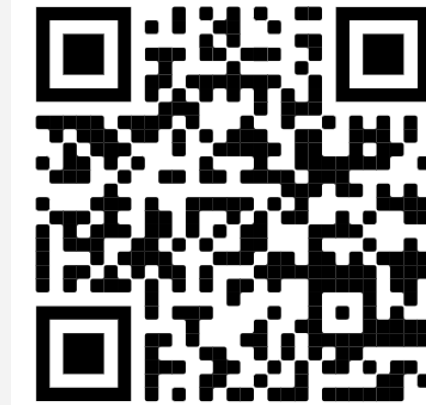
- No true uncertainty
- h^+ heuristic often performs poorly¹

1. Bonet and Geffner, “Planning as heuristic search,” in AI, 2001



SABRE

NARRATIVE PLANNER



<http://cs.uky.edu/~sgware/projects/sabre>

Background Music: <https://www.bensound.com>

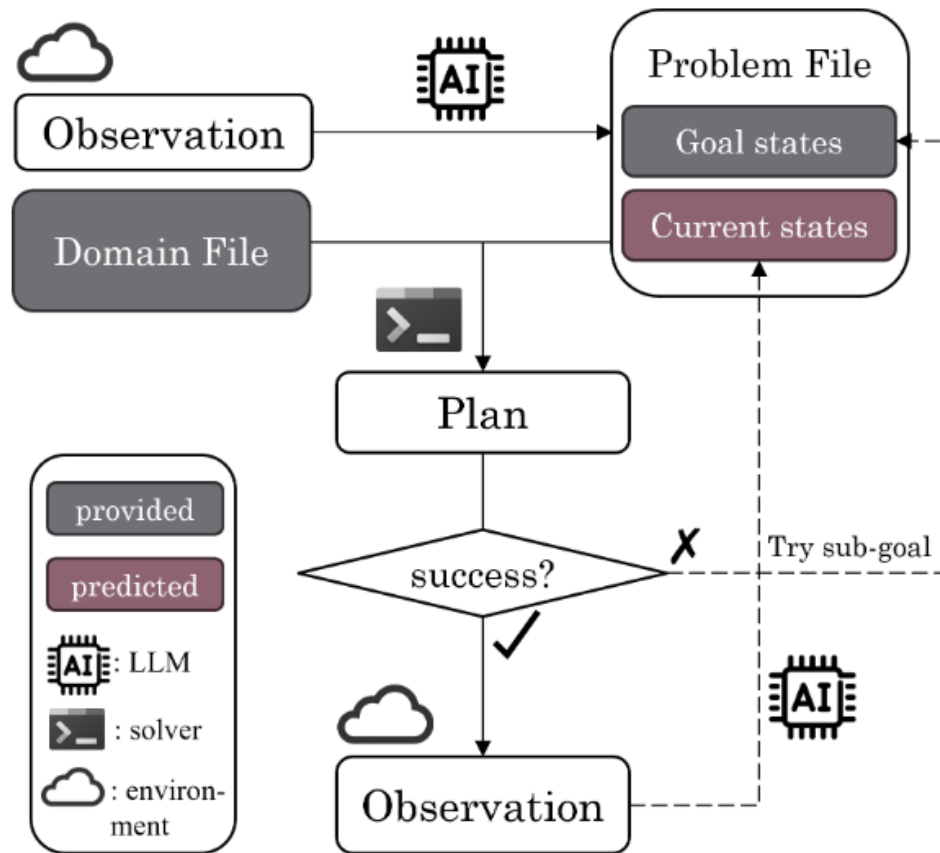
Neural Planning

Types of Neural Planning

Generation of planning language code

- Generating specification (e.g., domain & problem)
- To be run through planner

PDDLGO



Iterative creation of PDDL problem

Treat as partially-observed

Keep regenerating until plan succeeds

Types of Neural Planning

Generation of planning language code

- Generating specification (e.g., domain & problem)
- To be run through planner

Generation of planning-esque components

Neural Story Planning

Algorithm 1: Neural Plot Planner

```
1: Input: ending event sentence  $g$ ; Initial conditions  $I$ .
2: Initialize a plan  $P \leftarrow \emptyset$ ; Initialize  $queue \leftarrow \{g\}$ .
3: while  $queue \neq \emptyset$  do
4:   Let  $event \leftarrow \text{pop}(queue)$ 
5:   Let  $context \leftarrow$  sequence of events collected by running a breadth-first
     search from  $event$  to  $g$ .
6:   Let  $\Lambda \leftarrow$  all satisfied preconditions
7:   Let  $\Gamma \leftarrow \text{generate preconds}(event)$  for each character in  $event$ 
8:   if adding any precondition in  $\Gamma$  creates a cycle then
9:     remove  $event$  from  $P$ .
10:     $\Gamma \leftarrow$  unsatisfied preconditions due to removing  $event$ .
11:   for each  $c \in \Gamma$  do
12:     if  $c \in I$  or  $c$  meets conditions for not being expanded then
13:        $P \leftarrow P \cup \{nil \xrightarrow{c} event\}$  ▷ Dangling precondition
14:     else if there exists a precondition  $c' \in \Lambda$  that is similar to  $c$  then
15:        $event' \leftarrow$  event that satisfies  $c'$ 
16:        $P \leftarrow P \cup \{event' \xrightarrow{c'} event\}$  ▷ Reuse precondition
17:     else
18:        $event' \leftarrow \text{generate event}(c, context)$ 
19:        $P \leftarrow P \cup \{event' \xrightarrow{c} event\}$  ▷ Satisfy with new event
20:    $queue \leftarrow queue \cup \{event'\}$ 
```

Generates preconditions for characters

- Items needed
- Locations
- Item states
- Context (How)
- Interactions with others
- Reasons

Partial-order-planning–inspired generation

StoryVerse

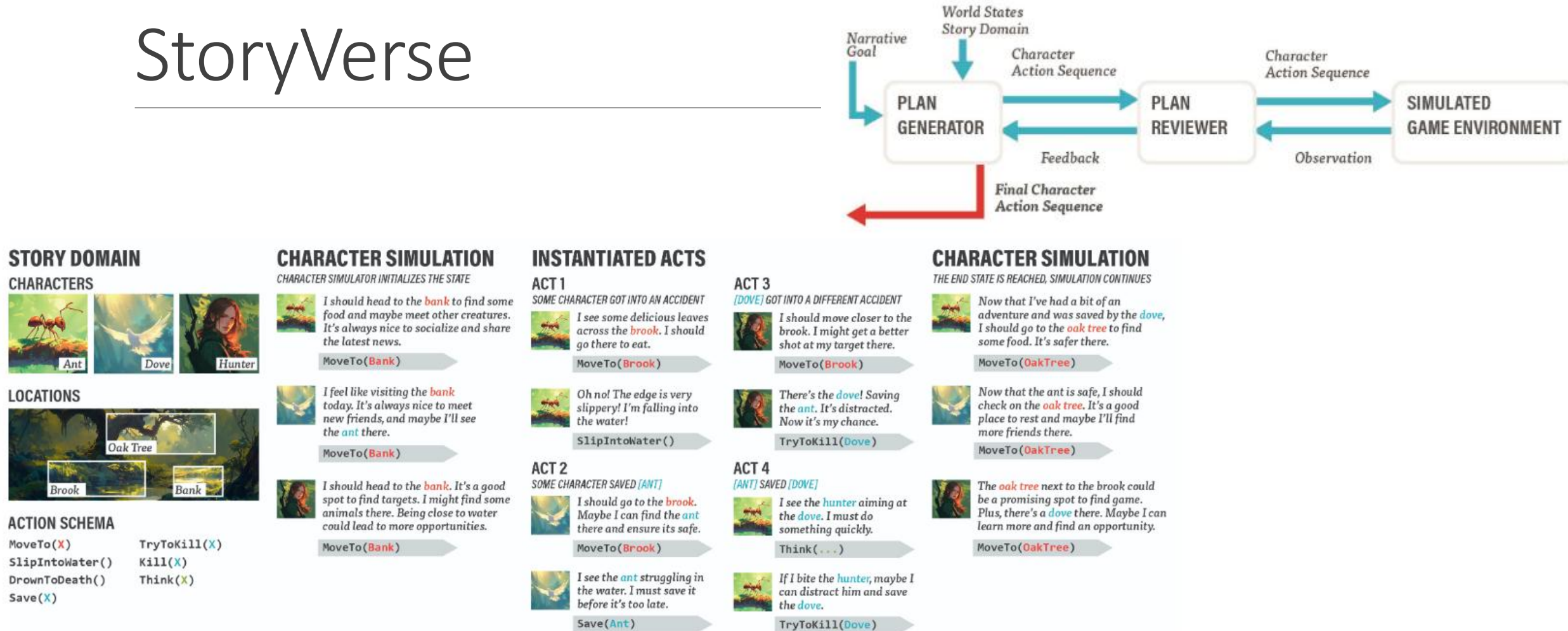


Figure 2: Two example story domains - The Ville (top) and Ant & Dove (bottom) - together with instantiated versions of the four abstract acts from Figure 1. Note that the text for narrations, dialogs, and monologues is all generated by LLMs.

Types of Neural Planning

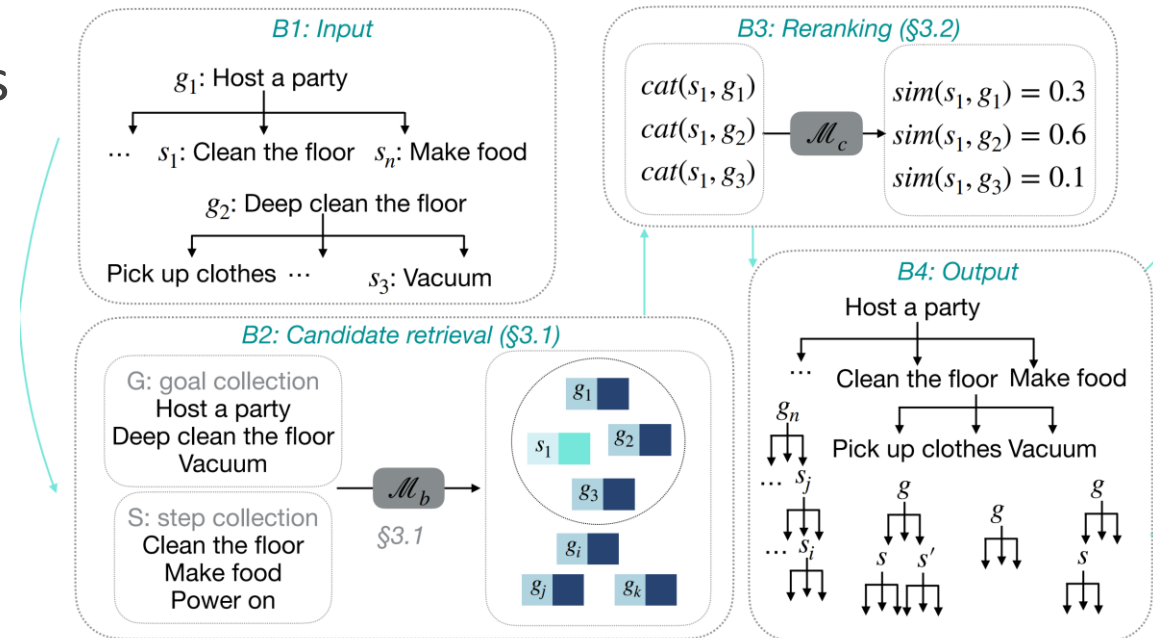
Generation of planning language code

- Generating specification (e.g., domain & problem)
- To be run through planner

Generation of planning-esque components

Use of LLM as *informal* planner

- Like the procedures work we looked at



Types of Neural Planning

Generation of planning language code

- Generating specification (e.g., domain & problem)
- To be run through planner

Generation of planning-esque components

Use of LLM as *informal* planner

- Like the procedures work we looked at
- Or using techniques like CoT

```
{"role": "system", "content": "# You are a helpful fiction writer assistant."},
{"role": "user", "content": f"{plot}\n
Summarize the plot above into a plot tree of
{'at most 6' if num_nodes == '' else num_nodes}
nodes with each node containing the state and goal of {char_name}, and the key
decision that propels the story forward. Each edge should contain a list of
events that lead {char_name} to the state of next node. Also, Given the same
state and goal of {char_name}, imagine an alternate decision that would have led
{char_name} to a different storyline. Output in JSON format with schema:
{JSON_SCHEMA}. Make sure that all important plot points are included in
'edgeEvents' but not in 'state'"
}
```

Table 5: Prompt for generating a tree from the plot (plot-to-tree).

Types of Neural Planning

Generation of planning language code

- Generating specification (e.g., domain & problem)
- To be run through planner

Generation of planning-esque components

Use of LLM as *informal* planner

- Like the procedures work we looked at
- Or using techniques like CoT
- Or guided/hierarchical generation like Plan & Write

Dynamic	Storyline	needed → money → computer → bought → happy
	Story	John <u>needed</u> a computer for his birthday. He worked hard to earn <u>money</u> . John was able to buy his <u>computer</u> . He went to the store and <u>bought</u> a computer. John was <u>happy</u> with his new computer.
Static	Storyline	computer → slow → work → day → buy
	Story	I have an old <u>computer</u> . It was very <u>slow</u> . I tried to <u>work</u> on it but it wouldn't work. One <u>day</u> , I decided to buy a new one. I <u>bought</u> a new computer .

Types of Neural Planning

Generation of planning language code

- Generating specification (e.g., domain & problem)
- To be run through planner

Generation of planning-esque components

Use of LLM as *informal* planner

- Like the procedures work we looked at
- Or using techniques like CoT
- Or guided/hierarchical generation like Plan & Write

Generation of formal plan

Provably Complete Planning

Complete = solve all instances of domain

Representing PDDL domains in Lean

- Lean is a programming language that can prove theorems

- 1) Stein et al. define planner components in Lean (e.g., state, goal checking) + Lean version of domain
- 2) GPT-5.6-Sol generates Lean code of plan based on PDDL domains
- 3) GPT generates (in Lean) proof that the plan works
- 4) Lean verifies proof

1) Lean encoding

```
structure State where
  statics : StaticState
  dynamic : DynamicState

def ActionPre ... (s : State) : Prop :=
  ...
```



2) Generalized Plan

```
def solve(s : State)(g : Goal) :
  List PlanAction :=
```



[Generalized Plan as Lean program]



3) Completeness Proof

```
theorem solveComplete
  (s : State)
  (g : Goal)
  (hvalid : ValidInstance s g):
  ValidPlan (solve s g) s ∧
  SatGoal (runPlan (solve s g) s) g := by
```



[Completeness Proof in Lean]



4) Kernel-check

```
#check (solveComplete :
  ∀ (s : State) (g : Goal),
  ValidInstance s g →
  ValidPlan (solve s g) s ∧
  SatisfiesGoal (runPlan (solve s g) s) g)
```

